EDUIGNITE

OPERATING SYSTEM

App Registry — Complete Manual

How founders install apps for schools,
and how developers add new ones.

Powered by EduIgnite · Bamenda, Cameroon

1. What the OS is

EduIgnite is built as a **Core** plus a set of **Apps**. Core is the part no school can run without: accounts, schools and classes, student records, notifications. Apps are the features sold and enabled on top of it — Attendance, Marks & Report Cards, Fees, Messaging, the Library, and the rest.

An app is a feature module that **ships inside the release**. Installing an app for a school grants that school a licence to use it. No code is transferred, downloaded or uploaded.

Why it works that way

EduIgnite runs on Android through Capacitor and on Windows through Electron. Both are compiled binaries that a school downloads. Schools also have to keep working through power and network cuts. A design where apps arrive over the network at runtime would break offline use, and would not survive app-store rules on shipping executable code. Shipping every app in the binary and controlling access with a licence is the only approach that is reliable on the connections these schools have.

What that means in practice

- A founder can give or remove an app for a school **instantly**. It is a licence change.
- A brand-new app requires a release: a backend deploy plus new mobile and desktop builds.
- Uninstalling an app **never deletes the school's data**.
- Core apps cannot be uninstalled by anyone. Nothing works without them.

Important. Hiding an app in the menu is presentation only. The real check runs on the server on every request, so a school without an app cannot reach its data even with a valid login and a direct request.

2. For founders: the Executive Console

The App Registry is open to the CEO, the CTO and Super Admins. Nobody else can see it, including school administrators.

Opening it

- Sign in to your founder account on web, desktop or mobile.
- In the left menu choose **App Registry**.
- Pick a school from the dropdown at the top right.

What you see

Every app in the release, as a card showing its name, what it does, its tier, and its state for the school you selected. Use the search box to filter by name or description.

Installing an app

- Find the app and press **Install for this school**.
- It is available to that school immediately — staff see it on their next screen load.
- No deploy, no restart, no new download for the school.

Removing an app

- Press **Remove from this school**.
- A confirmation appears listing exactly which records are affected.
- Confirm, and the school loses access at once.

> **Removal keeps the data.** If you remove Attendance, that school's attendance records are not deleted. Staff simply cannot open the app. Re-install it and everything is exactly where it was. This is deliberate: an accidental removal of a term's marks or attendance has to be recoverable.

Tiers

Each app sits in one of four tiers, which decides how a school gets it.

Tier	Meaning
Core	The platform itself. Always on, never billed, cannot be removed.
Included	Comes with any subscription. New schools get these automatically.
Standard	Sold separately. A founder installs it per school.
Premium	Sold separately, higher tier. A founder installs it per school.

Licence expiry

An installation can carry an expiry date. On the day after it passes, the school loses access automatically — no action needed from you — and the app shows as *Licence expired* in the console. Leave the date blank for no expiry.

New schools

When you create a school it is given every **Included** app straight away, so it is usable from the first login. Standard and Premium apps are added by you.

3. The apps in this release

24 apps ship in this release. This table is generated from the registry itself, so it always matches what is actually installed on the server.

App	Tier	What it does
Accounts & Sign-in	Core	Sign-in, password reset and student activation by matricule.
App Registry	Core	The catalogue of apps and which schools have them.
Notifications	Core	In-app and push notifications, mirrored to parents.
People	Core	Every account on the platform, their role and their profile.
Platform Settings	Core	Platform-wide settings, branding, fees and public content.
Schools & Classes	Core	Schools, sub-schools, classes, the timetable and school settings.
Student Records	Core	Student profiles, guardians, admission details and QR-verified ID cards.
AI Assistance	Premium	Study plans, performance insights, career reports and exam preparation.
Announcements	Included	School-wide announcements to staff, students and families.
App Showcase	Included	The catalogue of downloadable EduIgnite apps shown to schools.
Assignments	Standard	Set assignments with deadlines, collect and grade submissions.
Attendance	Included	Daily attendance per class, with absence alerts to parents.
Class Masters	Standard	Class-master assignments and teacher-class relationships.
Community & Blog	Standard	School stories, testimonies and the public blog.
Digital Library	Standard	Catalogue books, manage lending and chase overdue copies.
Exams	Standard	Schedule exams and run them online with automatic marking.
Feedback	Standard	Collect feedback from staff, students and parents.
Fees & Mobile Money	Standard	School fees, mobile-money collection, receipts and reminders.
Live Classes	Standard	Schedule and run live lessons, with the class notified.
Marks & Report Cards	Included	Sequence marks, averages and official bilingual Cameroon report cards.
Messaging	Standard	Direct and group chat for staff, students and parents.
Orders	Standard	Orders and platform purchases.
Staff Remarks	Standard	Private staff notes and conduct remarks on students.
Support	Included	Support tickets and contributions to the platform.

Core apps are listed first. Anything not marked Core can be installed and removed per school.

4. For developers: adding an app

An app is a Django app that declares itself, reads Core data through Core service APIs, and guards its own endpoints. Four steps.

Step 1 — Declare it

Create `apps/<name>/eduignite_app.py`:

```
from apps.registry.manifest import AppManifest, AppTier, register

register(
    AppManifest(
        key='transport',
        name='School Transport',
        summary='Bus routes, stops and pickup registers.',
        icon='Bus',
        tier=AppTier.STANDARD,
        roles=['SCHOOL_ADMIN', 'TEACHER', 'PARENT'],
        routes=['/dashboard/transport'],
        api_prefixes=['transport/'],
        core_services=['students'],
        owns_data=['bus routes', 'pickup records'],
        price_hint='Standard',
    )
)
```

Add the app to `INSTALLED_APPS`. The registry finds the declaration at startup. A mistake in it stops the server rather than producing a platform quietly missing a feature.

- **key** is permanent. Installations reference it; changing it orphans every licence.
- **routes** must be unique across all apps — two apps claiming one route is a test failure.
- **core_services** may only name Core apps.
- **owns_data** is shown verbatim in the uninstall warning, so write it in plain language.

Step 2 — Read Core data through the Core API

Never import another app's models. Student data goes through `apps.students.api`, which is enforced by a test. The full reference is in *docs/student-boundary.md*.

```
from apps.students import api as students_api

students = students_api.list_students(school=request.user.school)
```

Step 3 — Guard the endpoints

Every view the app owns must carry the guard:

```
from apps.registry.permissions import RequiresApp

class TransportViewSet(viewsets.ModelViewSet):
    permission_classes = [IsAuthenticated, RequiresApp('transport')]
```

> **Watch out.** If your class defines `get_permissions()`, DRF ignores `permission_classes` completely and your guard enforces nothing while still looking correct in review. Put the guard inside `get_permissions()` as well. A test checks for this, because it is invisible on inspection.

Step 4 — Run the tests

```
python manage.py test apps.registry core.tests.test_module_boundaries
```

These run without a database. They will fail if:

- an app has no name or summary, or a key that is not a slug;
- two apps claim the same route;
- an app depends on a service that is not Core;
- a guard is shadowed by `get_permissions()`;
- a sellable app guards no endpoint at all;
- anything outside `apps/students/` imports student models directly.

5. Reference

Endpoints

Endpoint	Who	Purpose
GET /api/v1/registry/my-apps/	Any signed-in user	The caller's apps and routes. Clients call this to build navigation.
GET /api/v1/registry/catalogue/	Founders	Every app in the release.
GET /api/v1/registry/schools/{id}/apps/	Founders	The catalogue annotated with one school's state.
POST /api/v1/registry/schools/{id}/apps/	Founders	Install, uninstall, or set a licence expiry.

Install / uninstall request

```
POST /api/v1/registry/schools/<school_id>/apps/
{
  "app_key": "attendance",
  "action": "install",   // install | uninstall | set_expiry
  "licence_expiry": "2026-09-01",  // optional, null for none
  "note": "Paid to end of year"
}
```

Service API

For code that needs to ask about entitlements directly:

Function	Returns
is_installed(school, app_key)	bool — may this school use the app
is_installed_for_user(user, app_key)	bool — same, via the user's school. Executives always pass.
installed_app_keys(school)	set of keys, always including Core
describe_for_school(school)	catalogue annotated with state, for the console
install(school, key, by=, licence_expiry=, note=)	AppInstallation
uninstall(school, key, by=, note=)	AppInstallation (data kept)
set_licence_expiry(school, key, expiry)	AppInstallation
install_defaults(school, by=)	int — installs every Included app

Where things live

Path	What
apps/registry/manifest.py	The manifest format and the registry itself
apps/registry/models.py	AppInstallation — one row per school per app
apps/registry/api.py	Service API; the supported way to ask and to change

Path	What
apps/registry/permissions.py	RequiresApp — server-side enforcement
apps/registry/views.py	Console endpoints
apps/registry/tests.py	Registry tests; no database needed
apps/<name>/eduignite_app.py	Each app's declaration
src/app/dashboard/apps/page.tsx	The Executive Console screen
src/lib/hooks/useRegistry.ts	Client hooks
docs/student-boundary.md	The Core service manual

6. Deploying this release

This release adds the registry to a platform where every school already had every feature. The migration handles that, but the order matters.

Order of operations

- Deploy the backend and run `python manage.py migrate`.
- Migration `registry.0002` grants every existing school every non-core app.
- Restart gunicorn **and** the Celery worker and beat.
- Sign in as a founder and open App Registry to confirm schools show their apps.

Do not skip the migration. Without it, entitlements are enforced that nobody has been granted, and every existing school loses Attendance, Marks, Fees and Chat the moment the release goes live. With it, the registry changes nothing for current schools and starts mattering only for new ones.

Restarting Celery

The worker and beat load application code at startup. Restarting only gunicorn leaves them running the previous release against the new API, which has caused a worker restart loop on this platform before. Restart all three.

Verifying after deploy

- Open App Registry and check an existing school lists its apps as installed.
- Remove a Standard app from a test school; confirm it disappears from that school's menu.
- Confirm the endpoint refuses it too, not just the menu.
- Re-install it and confirm the school's existing records are all still there.
- Create a new school and confirm it arrives with the Included apps.

Known gap. The registry's own tests run without a database and all pass. The paths that touch the database — installing, removing, the grandfather migration — have not been executed against a real database, because the test database was not reachable from the machine this was built on. Run the checks above on a staging copy before trusting this in front of schools.